Full Stack Web Development Lab Record

List of Experiments

S.No	Description of the Experiment	Course Outcome
1	Design static webpage using HTML components	CO1
2	Design webpage using HTML5 & CSS3	CO1
3	Write a program using JavaScript (arrays, functions)	CO1
4	Develop dynamic webpage using JavaScript	CO1
5	Implement modules in Node.js	CO2
6	Develop mini application using Express and Node.js	CO2
7	Create functional and class components using React	CO3
8	Implement HTTP methods using JSON Server	CO3
9	Develop CRUD operations using MongoDB (Mongoose and mongosh)	CO4
10	Develop main project using Express, React and MongoDB	CO4

Experiment 1: Design Static Webpage using HTML Components

Aim: To design a static webpage using basic HTML components such as headings, paragraphs, lists, images, links, tables and forms.

Theory: HTML (HyperText Markup Language) describes the structure of a webpage using elements (tags). Static webpages display fixed content and do not change based on user interaction. Common components include headings (`<h1>`–`<h6>`), paragraphs (`<p>`), lists (`<ul>`, `<ol>`), links (`<a>`), images (`<img>`), tables (`<table>`) and forms (`<form>`).

Key concepts:

- **Document skeleton:** `<!DOCTYPE html>` declares the document type. `<html>` is the root, `<head>` holds metadata (character set, title) that the browser does not display, and `<body>` holds everything visible on the page.
- **Block vs inline elements:** Block elements (`<h1>`, `<p>`, `<ul>`, `<table>`, `<form>`) start on a new line and take the full width. Inline elements (`<a>`, `<span>`, `<input>`) sit within a line of text.
- **Lists:** `<ol>` creates a numbered list and `<ul>` creates a bulleted list. Each item is wrapped in `<li>`.
- **Tables:** `<table>` is the container, `<tr>` a row, `<th>` a header cell and `<td>` a data cell. Tables are meant for tabular data, not for page layout.
- **Forms:** `<form>` groups input controls. The `type` attribute of `<input>` selects the control (`text`, `email`, `password`, `number`, `date`). `<label>` associates a caption with its input, and `name` identifies the value when the form is submitted.

Element	Purpose	Example
`<h1>` to `<h6>`	Headings, `<h1>` is the most important	`<h1>Title</h1>`
`<p>`	Paragraph	`<p>Some text</p>`
`<a href="">`	Hyperlink	`<a href="https://nodejs.org">Node.js</a>`
`<img src="" alt="">`	Image with alternative text	`<img src="logo.png" alt="Logo">`
`<table>`, `<tr>`, `<td>`	Tabular data	`<td>CSE</td>`
`<form>`, `<input>`, `<button>`	User input	`<input type="email">`

Procedure:

1. Open a text editor (VS Code, Notepad++ or similar) and create a new file named `exp1.html`.
2. Type the document skeleton: `<!DOCTYPE html>`, `<html>`, `<head>` with the title, and `<body>`.
3. Add the headings, paragraph, ordered list, unordered list with links, table and form, in that order.
4. Save the file and open it in a browser by double-clicking it, or by dragging it into the browser window.
5. Check that every element appears as expected. Use View Page Source to confirm the markup.

Code (exp1.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Static Webpage</title>
</head>
<body>
  <h1>Welcome to My Static Webpage</h1>
  <h3>Web Technologies Lab - Experiment 1</h3>
  <p>This page is built only with basic HTML components.</p>

  <h2>Favourite Subjects</h2>
  <ol>
    <li>Data Structures</li>
    <li>Database Management Systems</li>
    <li>Web Technologies</li>
  </ol>

  <h2>Useful Links</h2>
  <ul>
    <li><a href="https://developer.mozilla.org">MDN Web Docs</a></li>
    <li><a href="https://nodejs.org">Node.js</a></li>
  </ul>

  <h2>Student Details</h2>
  <table border="1">
    <tr><th>Name</th><th>Roll No</th><th>Branch</th></tr>
```

```
    <tr><td>Student One</td><td>23XX1A0501</td><td>CSE</td></tr>
  </table>

  <h2>Contact Form</h2>
  <form>
    <label>Name: <input type="text" name="name"></label><br><br>
    <label>Email: <input type="email" name="email"></label><br><br>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

Output: The browser displays a main heading, a subheading, a paragraph, a numbered list of subjects, a bulleted list of two clickable links, a bordered table with one student record, and a form with Name and Email fields and a Submit button.



Figure: Experiment 1 - Static webpage rendered in browser

Result: A static webpage using HTML components was designed and verified in the browser.

Viva questions:

1. What is the difference between a block-level and an inline element? Give one example of each.
2. Why is `alt` required on an `<img>` element?
3. What is the role of the `label` element in a form?
4. Why should tables not be used for page layout?

Experiment 2: Design Webpage using HTML5 & CSS3

Aim: To design a webpage using HTML5 semantic elements and style it using CSS3.

Theory: HTML5 introduced semantic elements such as `<header>`, `<nav>`, `<section>`, `<article>`, `<aside>` and `<footer>`, which describe the meaning of content. CSS3 controls presentation: colours, fonts, layout, spacing and responsiveness. Styles can be placed inline, in an internal `<style>` block, or in an external stylesheet. External stylesheets are preferred because they separate content from presentation.

Key concepts:

- **Semantic elements:** `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` and `<footer>` tell the browser, search engines and screen readers what each part of the page means. A `<div>` has no meaning and is used only for grouping.
- **CSS syntax:** A rule has a selector and a declaration block, for example `header { background-color: #2c3e50; }`. Selectors can target an element (`nav`), a class (`.card`), an id (`#home`), a descendant (`nav a`) or a state (`nav a:hover`).
- **Cascade and specificity:** When several rules match the same element, the browser chooses the rule with the highest specificity. Inline styles beat id selectors, which beat class selectors, which beat element selectors. Later rules win when specificity is equal.
- **Box model:** Every element is a box made of content, padding, border and margin. `padding` is the space inside the border and `margin` is the space outside it.
- **Flexbox:** `display: flex` arranges children in a row (default) or column. `justify-content` aligns items along the main axis and `gap` sets the space between them. The navigation bar in this experiment uses Flexbox.
- **Responsive design:** The viewport meta tag makes the page scale on phones. The `@media (max-width: 600px)` rule applies new styles only when the screen is narrow, so the navigation stacks vertically.
- **External stylesheets:** `<link rel="stylesheet" href="style.css">` loads the styles separately, so the same stylesheet can be shared by many pages.

CSS property	Effect
`color`, `background-color`	Text and background colours
`font-family`, `font-weight`	Typeface and boldness
`padding`, `margin`	Space inside and outside the box
`border-radius`, `box-shadow`	Rounded corners and drop shadows
`display: flex`, `justify-content`, `gap`	Flexbox layout

Procedure:

1. Create a folder, for example `exp2`, and place `index.html` and `style.css` inside it.
2. Write the HTML structure with the semantic elements in the order header, nav, main, aside and footer.
3. Write the CSS rules for each element, starting with `body` and working down.
4. Open `index.html` in a browser, then resize the window below 600 px to confirm the media query.
5. Hover over a navigation link to check the colour change.

Code (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML5 & CSS3 Webpage</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>My Portfolio</h1>
  </header>

  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="home">
      <h2>Home</h2>
      <p>Welcome to my styled webpage built using HTML5 and CSS3.</p>
```

```
    </section>
    <article id="about">
      <h2>About</h2>
      <p>I am a third-year engineering student interested in web development.</p>
    </article>
  </main>

  <aside>
    <p>Tip: Resize the browser to see the responsive layout.</p>
  </aside>

  <footer>
    <p>&copy; 2026 My Portfolio</p>
  </footer>
</body>
</html>
```

Code (style.css):

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f4f4f4;
}
header {
  background-color: #2c3e50;
  color: white;
  text-align: center;
  padding: 20px;
}
nav {
  background-color: #34495e;
  display: flex;
  justify-content: center;
  gap: 20px;
  padding: 10px;
}
nav a {
  color: white;
  text-decoration: none;
  font-weight: bold;
}
nav a:hover {
  color: #f1c40f;
}
main {
  padding: 20px;
}
section, article {
  background: white;
  padding: 15px;
  margin-bottom: 15px;
  border-radius: 8px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}
aside {
  background-color: #ecf0f1;
  margin: 0 20px;
  padding: 10px;
  border-left: 4px solid #f1c40f;
}
footer {
  background-color: #2c3e50;
  color: white;
  text-align: center;
  padding: 10px;
  margin-top: 20px;
}
@media (max-width: 600px) {
  nav {
    flex-direction: column;
    align-items: center;
  }
}
```

Output: The page shows a dark blue header, a navigation bar with three links (yellow on hover), two white cards for Home and About with rounded corners and shadows, a light aside box with a yellow left border, and a dark footer. On narrow screens the navigation links stack vertically.

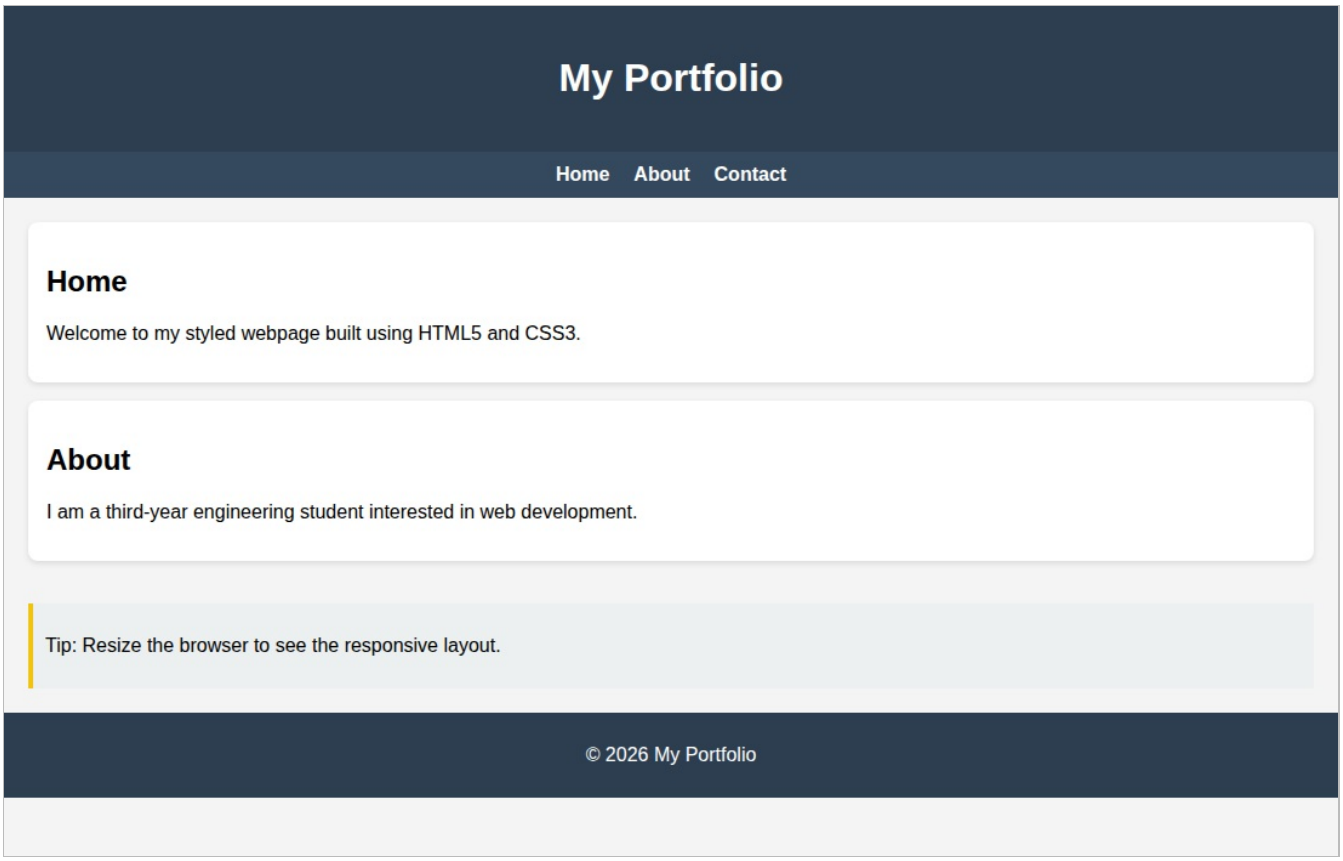


Figure: Experiment 2 - HTML5 and CSS3 responsive webpage

Result: A responsive webpage was designed using HTML5 semantic elements and styled with CSS3.

Viva questions:

1. Why are semantic elements preferred over generic `<div>` elements?
2. What is the difference between `padding` and `margin`?
3. How does specificity decide which CSS rule applies?
4. What does the viewport meta tag do on a mobile device?

Experiment 3: Write a Program using JavaScript (Arrays, Functions)

Aim: To write JavaScript programs that use arrays and user-defined functions.

Theory: An array is an ordered collection of values accessed by index. JavaScript provides methods such as `push()`, `pop()`, `map()`, `filter()` and `reduce()`. A function is a reusable block of code defined with the `function` keyword or as an arrow function. Functions may accept parameters and return values.

Key concepts:

- **Arrays:** An array stores values in index order, starting at 0. `arr.length` gives the number of elements. Arrays can hold mixed types, but a single typed collection is easier to process.
- **Common array methods:**

Method	Purpose	Modifies original?
`push()` / `pop()`	Add or remove at the end	Yes
`unshift()` / `shift()`	Add or remove at the start	Yes
`slice()`	Copy a portion of the array	No
`reverse()`	Reverse order	Yes
`map()`	Transform each element and return a new array	No
`filter()`	Keep elements that satisfy a condition	No
`reduce()`	Combine all elements into one value	No

- **Functions:** Declared with `function name(params) { ... }` or as arrow functions such as `n => n * n`. Parameters are local to the function, and `return` sends a value back to the caller.
- **Copying arrays:** `arr.slice().reverse()` reverses a copy and leaves the original unchanged, which is why `reverseArray` does not affect `numbers`.
- **Prime check:** `isPrime` tests divisors only up to the square root of `n`, because any factor larger than the square root pairs with one smaller than it. This makes the test run in about O(√n) time.

Procedure:

1. Create a file `exp3.js` in a folder.
2. Define the four functions: `sumArray`, `findMax`, `reverseArray` and `isPrime`.
3. Declare the `numbers` array and call each function, printing the results with `console.log`.
4. Use `filter` with `isPrime` and `map` with an arrow function to produce the prime and square lists.
5. Run the file with `node exp3.js` and compare the output with the expected values.

Code (exp3.js):

```
// Function to find the sum of array elements
function sumArray(arr) {
  let total = 0;
  for (let i = 0; i < arr.length; i++) {
    total += arr[i];
  }
  return total;
}

// Function to find the largest element
function findMax(arr) {
  let max = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
      max = arr[i];
    }
  }
  return max;
}

// Function to reverse an array
function reverseArray(arr) {
  return arr.slice().reverse();
}

// Function to check whether a number is prime
function isPrime(n) {
```

```
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) return false;
  }
  return true;
}

// Main program
let numbers = [12, 45, 7, 23, 56, 89, 3];
console.log("Original array:", numbers);
console.log("Sum of elements:", sumArray(numbers));
console.log("Largest element:", findMax(numbers));
console.log("Reversed array:", reverseArray(numbers));

let primes = numbers.filter(isPrime);
console.log("Prime numbers:", primes);

let squares = numbers.map(n => n * n);
console.log("Squares:", squares);
```

Output:

```
Original array: [ 12, 45, 7, 23, 56, 89, 3 ]
Sum of elements: 235
Largest element: 89
Reversed array: [ 3, 89, 56, 23, 7, 45, 12 ]
Prime numbers: [ 7, 23, 89, 3 ]
Squares: [ 144, 2025, 49, 529, 3136, 7921, 9 ]
```

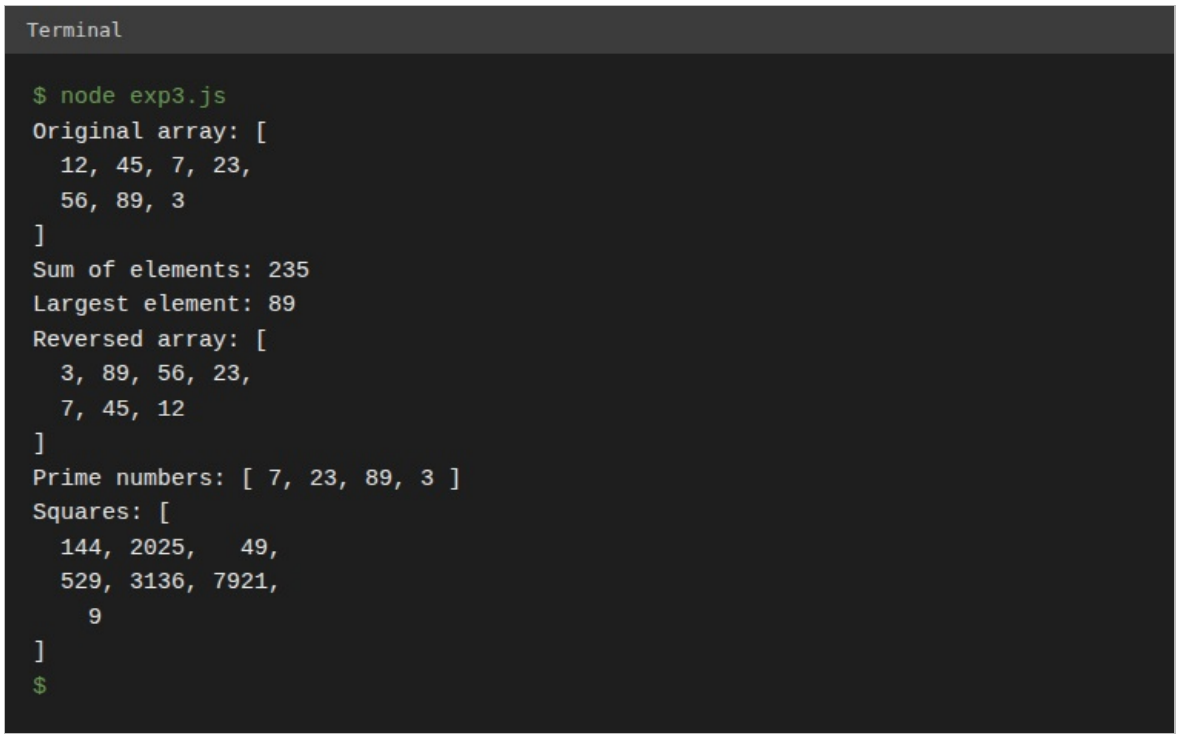


Figure: Experiment 3 - Terminal output of exp3.js

Result: JavaScript programs using arrays and functions were written and executed successfully.

Viva questions:

1. What is the difference between `map()` and `filter()`?
2. Why does `arr.slice().reverse()` not change the original array?
3. Why does the prime check stop at the square root of `n`?
4. What is the difference between a function declaration and an arrow function?

Experiment 4: Develop Dynamic Webpage using JavaScript

Aim: To develop a dynamic webpage that responds to user events and modifies the DOM using JavaScript.

Theory: The Document Object Model (DOM) represents an HTML document as a tree of objects. JavaScript can access elements using `document.getElementById()` and `querySelector()`, change their content or style, create new elements, and respond to events like `click`, `mouseover` and `input`. This makes the page dynamic without reloading.

Key concepts:

- **DOM tree:** The browser converts the HTML into a tree of nodes. JavaScript can read and change any node, and the page updates immediately without a reload.
- **Selecting elements:**

Method	Returns
`document.getElementById("id")`	One element with that id
`document.querySelector("css-selector")`	First element matching a CSS selector
`document.querySelectorAll("css-selector")`	All matching elements as a list

- **Events:** An event is an action such as `click`, `mouseover`, `mouseout`, `input` or `submit`. Handlers can be attached inline (`onclick="changeMessage()"`) or with `addEventListener`, which is the preferred method because it keeps HTML and JavaScript separate.
- **Changing content:** `innerHTML` parses a string as HTML, while `textContent` inserts plain text. `textContent` is safer when the value comes from a user, because it cannot inject markup.
- **Creating elements:** `document.createElement("li")` creates a node, `textContent` sets its text, and `appendChild` attaches it to a parent. This is how the item list grows.
- **Input validation:** `input.value.trim() === ""` checks that the field is not empty or only spaces before the item is added.

Procedure:

1. Create `exp4.html` with a heading, a paragraph, two buttons, a box, an input and an empty list.
2. Add the CSS in a `<style>` block for the layout and the box size.
3. Write the functions `changeMessage`, `changeColor` and `addItem` in a `<script>` block.
4. Attach `mouseover` and `mouseout` listeners to the box.
5. Open the page, test each button, hover over the box, and try adding an empty item to see the alert.

Code (exp4.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dynamic Webpage</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center; }
    #box { width: 200px; height: 100px; margin: 20px auto; background: lightblue; line-height: 100px; }
    li { margin: 5px; }
  </style>
</head>
<body>
  <h1 id="title">Dynamic Webpage</h1>
  <p id="message">Click the buttons below.</p>
  <button onclick="changeMessage()">Change Message</button>
  <button onclick="changeColor()">Change Colour</button>

  <div id="box">Hover over me</div>

  <input type="text" id="itemInput" placeholder="Enter an item">
  <button onclick="addItem()">Add Item</button>
  <ul id="itemList"></ul>

  <script>
    function changeMessage() {
      document.getElementById("message").innerHTML = "The message has been changed!";
    }

    function changeColor() {
      let colors = ["lightcoral", "lightgreen", "gold", "plum"];
      let random = colors[Math.floor(Math.random() * colors.length)];
      document.getElementById("title").style.color = random;
```

```
    }

    const box = document.getElementById("box");
    box.addEventListener("mouseover", () => box.style.background = "orange");
    box.addEventListener("mouseout", () => box.style.background = "lightblue");

    function addItem() {
      const input = document.getElementById("itemInput");
      if (input.value.trim() === "") {
        alert("Please enter an item.");
        return;
      }
      const li = document.createElement("li");
      li.textContent = input.value;
      document.getElementById("itemList").appendChild(li);
      input.value = "";
    }
  </script>
</body>
</html>
```

Output: On loading, the page shows the title, a message, two buttons, a light blue box and an input field. "Change Message" replaces the paragraph text, "Change Colour" changes the title colour randomly, hovering over the box turns it orange, and "Add Item" appends the entered text as a list item. An empty input shows an alert.

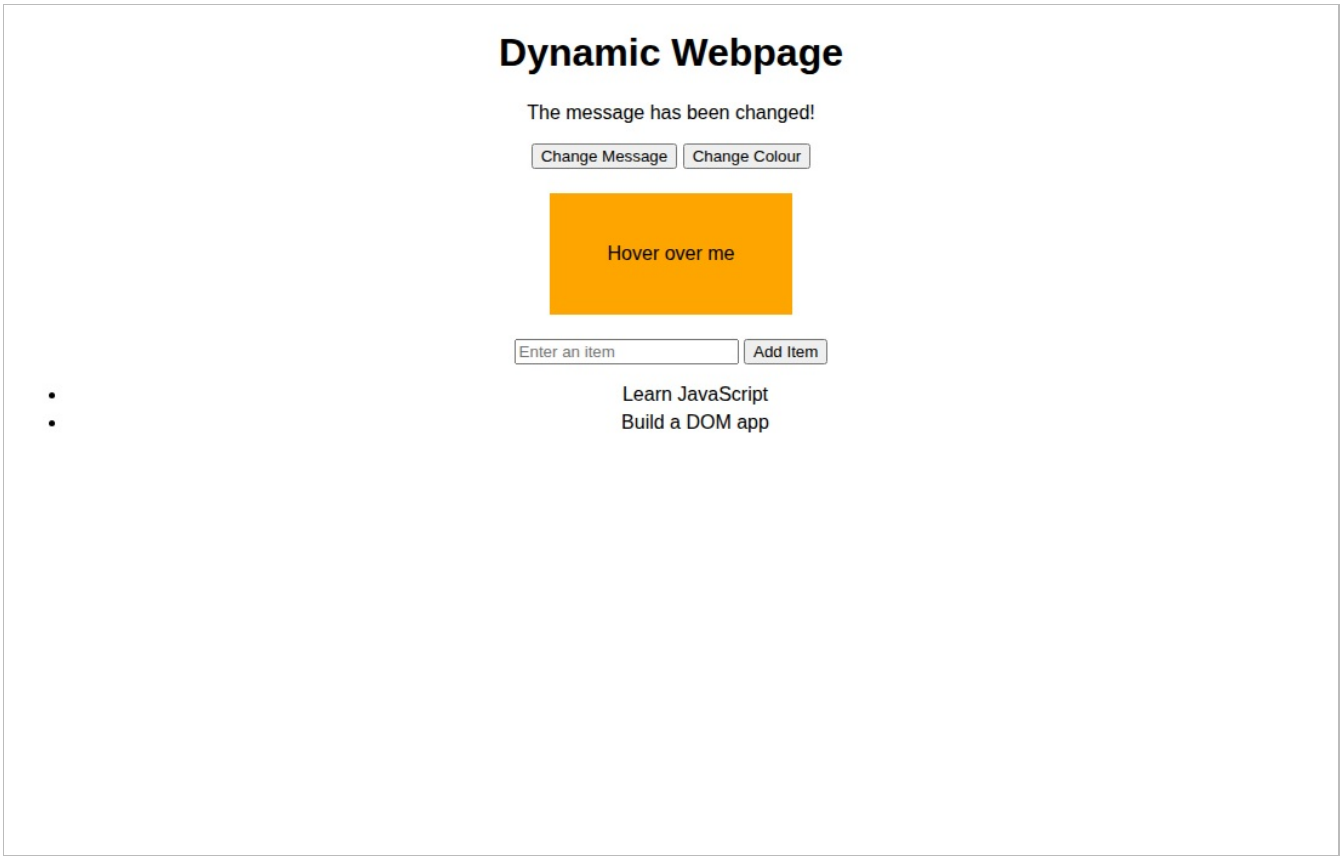


Figure: Experiment 4 - Dynamic webpage after interaction

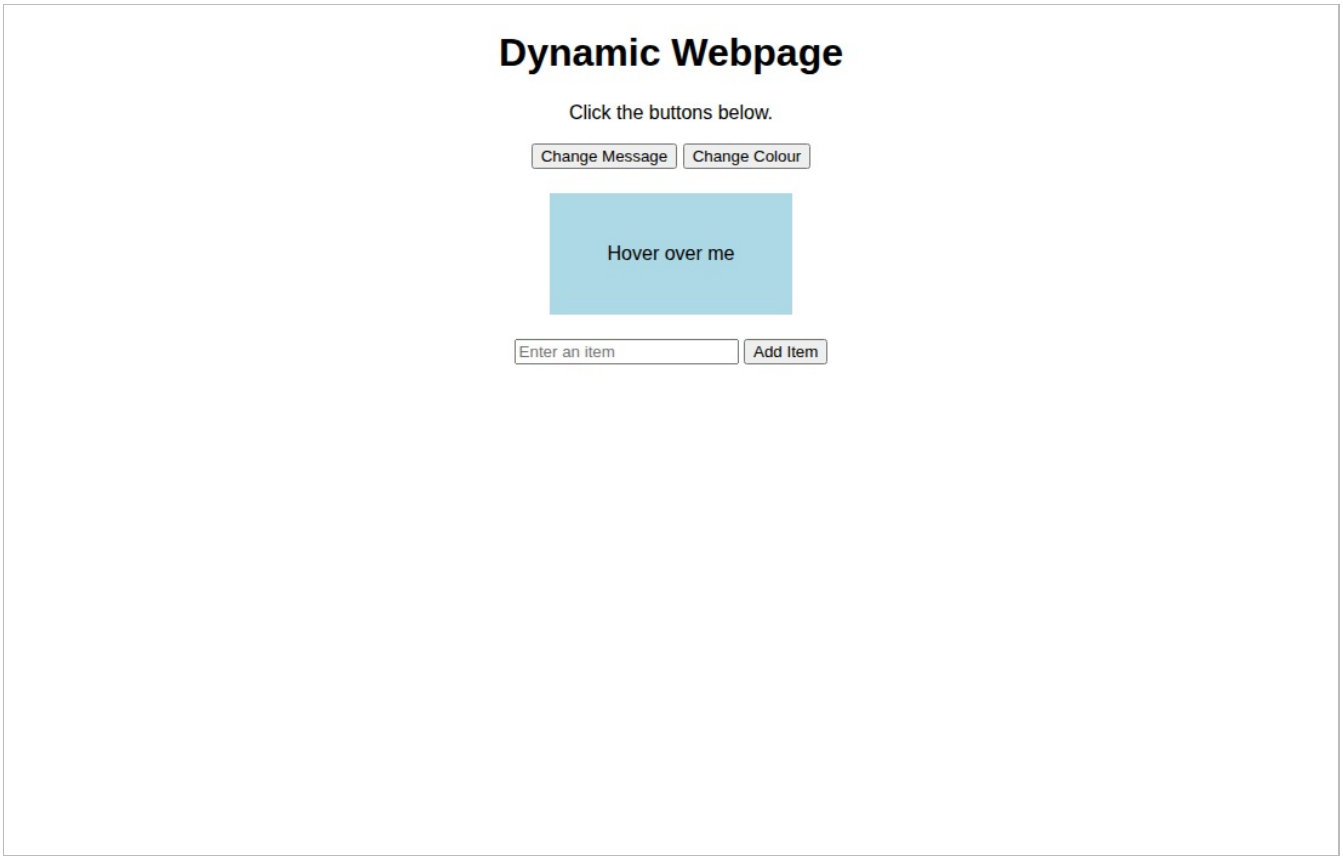


Figure: Experiment 4 - Dynamic webpage on initial load

Result: A dynamic webpage responding to user events was developed using JavaScript and the DOM.

Viva questions:

1. What is the DOM?
2. What is the difference between `innerHTML` and `textContent`?
3. Why is `addEventListener` preferred over inline `onclick` attributes?
4. How does `appendChild` differ from `innerHTML` when adding list items?

Experiment 5: Implement Modules in Node.js

Aim: To create and use custom and built-in modules in Node.js.

Theory: Node.js follows the CommonJS module system. Each file is a module. A module exposes its functions or variables using `module.exports` and another file imports them with `require()`. Node.js also provides built-in modules such as `fs`, `path`, `os` and `http`, which are imported the same way without installation.

Key concepts:

- **CommonJS:** Node.js treats each file as a module with its own scope. Variables are private unless exported with `module.exports`. Imports use `require()`, and a module is loaded only once and then cached.
- **Relative and package imports:** `require("./mathUtils")` loads a local file, while `require("express")` loads an installed npm package from `node_modules`.
- **ES modules:** Modern JavaScript also supports `import` and `export`. Node.js uses CommonJS by default, and ES modules are enabled with `"type": "module"` in `package.json` or the `.mjs` extension.
- **Built-in modules:**

Module	Use
`fs`	Read, write and delete files
`path`	Build and parse file paths in a cross-platform way
`os`	Information about the operating system
`http`	Create web servers and make HTTP requests
`events`	Event emitter used by most Node.js APIs

- **Synchronous vs asynchronous:** `writeFileSync` and `readFileSync` block until the operation finishes. Asynchronous versions take a callback or return a promise and are preferred in servers so the program is not stalled.
- **Globals:** `__filename` gives the full path of the current file and `__dirname` gives its folder.

Procedure:

1. Create `mathUtils.js` and write the three functions, then export them with `module.exports`.
2. Create `app.js` and import `mathUtils` with `require("./mathUtils")`.
3. Import the built-in modules `fs`, `os` and `path`.
4. Call each function and print the results, then write and read `sample.txt`.
5. Run `node app.js` and check the output and the new file in the folder.

Code (mathUtils.js):

```
// Custom module
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

function greet(name) {
  return `Hello, ${name}! Welcome to Node.js modules.`;
}

module.exports = { add, multiply, greet };
```

Code (app.js):

```
// Importing a custom module
const mathUtils = require("./mathUtils");

// Importing built-in modules
const fs = require("fs");
const os = require("os");
const path = require("path");

console.log("Addition: ", mathUtils.add(10, 5));
console.log("Multiplication: ", mathUtils.multiply(4, 6));
console.log(mathUtils.greet("Student"));
```

```
// Using built-in modules
console.log("Operating System:", os.platform());
console.log("File name of this script:", path.basename(__filename));

fs.writeFileSync("sample.txt", "This file was created using the fs module.");
console.log("File content:", fs.readFileSync("sample.txt", "utf8"));
```

Output:

```
Addition:  15
Multiplication:  24
Hello, Student! Welcome to Node.js modules.
Operating System: linux
File name of this script: app.js
File content: This file was created using the fs module.
```

(The OS value depends on the machine, for example `win32` on Windows or `darwin` on macOS.)



Figure: Experiment 5 - Terminal output of app.js

Result: Custom and built-in Node.js modules were created, imported and used successfully.

Viva questions:

1. What is the difference between `module.exports` and `exports`?
2. Why does `require("./mathUtils")` need the `./` prefix?
3. What is the difference between `readFileSync` and `readFile`?
4. What does `__dirname` store?

Experiment 6: Develop Mini Application using Express and Node.js

Aim: To develop a small web application using the Express framework on Node.js.

Theory: Express is a minimal and flexible Node.js web application framework. It provides routing, middleware support and easy handling of HTTP requests and responses. Routes map URLs and HTTP methods to handler functions. Middleware such as `express.json()` parses incoming request bodies.

Key concepts:

- **Application and routes:** `app.get`, `app.post` and `app.delete` register handlers for an HTTP method and a URL path. Path parameters such as `:id` are read from `req.params`.
- **Request and response:** `req.body` holds parsed JSON data (after `express.json()`), and `res.json()` or `res.send()` sends the response. `res.status(201)` sets the status code.
- **Middleware:** A function that runs before the route handler. `express.json()` parses JSON bodies and `express.static("public")` serves files from the `public` folder.
- **HTTP status codes used:**

Code	Meaning	Where used
200 OK	Request succeeded	GET, DELETE
201 Created	New resource created	POST
404 Not Found	Resource does not exist	Missing route or record
500 Internal Server Error	Unexpected server failure	Unhandled exceptions

- **REST style:** Resources are nouns in the URL (`/api/tasks`) and the method describes the action: GET reads, POST creates, DELETE removes.
- **Limitation of this code:** Tasks are kept in a variable in memory, so they are lost when the server stops. Also, `tasks.length + 1` can produce duplicate ids after a delete. A database such as MongoDB (Experiment 9) solves both problems.

Procedure:

1. Create the project folder, run `npm init -y` and `npm install express`.
2. Write `server.js` with the routes shown above.
3. Run `node server.js` and open `http://localhost:3000` in a browser.
4. Open `/api/tasks` to see the list in JSON.
5. Send a POST request with a title, using Postman or `curl`, and then check the list again.

Setup:

```
mkdir express-mini-app
cd express-mini-app
npm init -y
npm install express
```

Code (server.js):

```
const express = require("express");
const app = express();
const PORT = 3000;

app.use(express.json());
app.use(express.static("public"));

// Home route
app.get("/", (req, res) => {
  res.send("<h1>Welcome to the Express Mini Application</h1><a href='/api/tasks'>View Tasks</a>");
});

// In-memory task list
let tasks = [
  { id: 1, title: "Learn Node.js", done: true },
  { id: 2, title: "Build an Express app", done: false }
];

// Get all tasks
app.get("/api/tasks", (req, res) => {
  res.json(tasks);
});
```

```
// Add a task
app.post("/api/tasks", (req, res) => {
  const task = { id: tasks.length + 1, title: req.body.title, done: false };
  tasks.push(task);
  res.status(201).json(task);
});

// Delete a task
app.delete("/api/tasks/:id", (req, res) => {
  tasks = tasks.filter(t => t.id !== Number(req.params.id));
  res.json({ message: "Task deleted" });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Output:

Running `node server.js` prints:

```
Server running at http://localhost:3000
```

Opening `http://localhost:3000/api/tasks` in the browser shows:

```
[
  { "id": 1, "title": "Learn Node.js", "done": true },
  { "id": 2, "title": "Build an Express app", "done": false }
]
```

A POST request with body `{"title":"Write report"}` returns the new task with status 201.

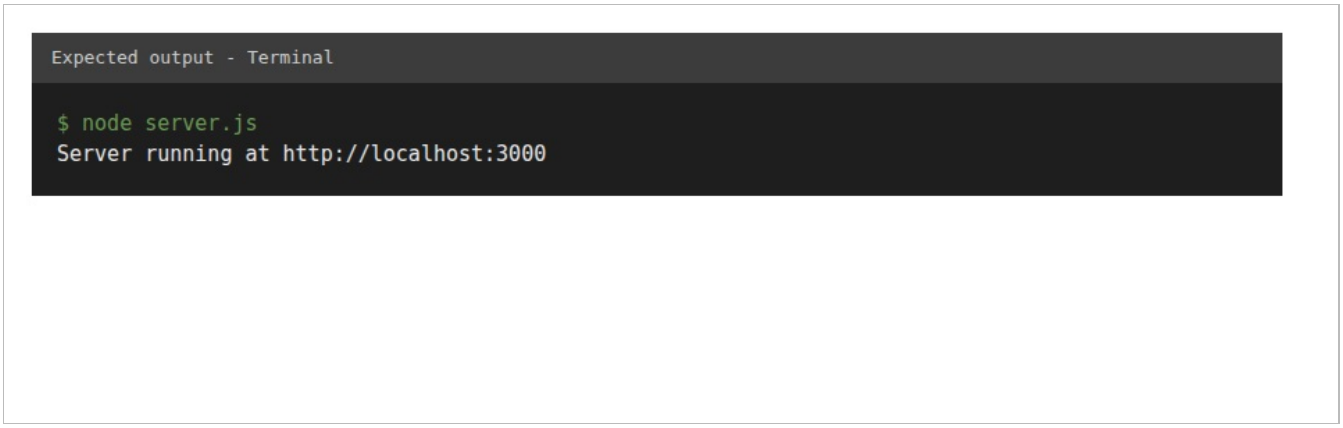


Figure: Experiment 6 - Terminal output (server started)



Figure: Experiment 6 - Browser output of /api/tasks

Result: A mini application with routing and REST-style endpoints was developed using Express and Node.js.

Viva questions:

1. What is middleware in Express?

2. What is the difference between `req.params` and `req.body`?
3. Why is status 201 returned for a POST request?
4. What is the main limitation of storing tasks in an array?

Experiment 7: Create Functional and Class Components using React

Aim: To create functional and class components in React and understand props and state.

Theory: React builds user interfaces from reusable components. A functional component is a JavaScript function that returns JSX and uses Hooks such as `useState` for state. A class component extends `React.Component`, keeps state in `this.state` and updates it with `this.setState()`. Props pass data from parent to child components.

Key concepts:

- **JSX:** A syntax that looks like HTML inside JavaScript. It is compiled to function calls, so `{count}` inserts the value of a variable and `{...}` can contain any expression.
- **Components:** A component is a function (or class) that returns JSX. `Greeting` is a reusable component that receives data through props.
- **Props:** Read-only inputs passed from parent to child, for example `<Greeting name="Student" />`. A child must never change its own props.
- **State:** Data that can change over time and cause the component to re-render. `useState(0)` returns the current value and a setter function. Calling the setter schedules a re-render with the new value.
- **Hook rules:** Hooks must be called at the top level of a component, never inside loops or conditions, and their names must start with `use`.
- **Class components:** Store state in `this.state`, update it with `this.setState()`, and must define a `render()` method. `setState` merges the new values into the existing state object.

Feature	Functional component	Class component
Declared as	A function	A class extending `Component`
State	`useState` hook	`this.state` and `this.setState()`
Lifecycle	`useEffect` hook	`componentDidMount`, `componentDidUpdate`
Modern usage	Preferred	Legacy, still supported

- **Virtual DOM:** React compares the new output with the previous output and updates only the changed parts of the real DOM.
- **Vite:** A fast development server and build tool. `npm run dev` starts it, usually at `http://localhost:5173`.

Procedure:

1. Create the project with `npm create vite@latest`, select React, then run `npm install` and `npm run dev`.
2. Create `FunctionalComponent.jsx` with `Greeting` and `FunctionalCounter`.
3. Create `ClassComponent.jsx` with the class counter.
4. Import both in `App.jsx` and render them.
5. Click each Increment button three times and confirm that the two counters change independently.

Setup:

```
npm create vite@latest react-components -- --template react
cd react-components
npm install
npm run dev
```

Code (src/FunctionalComponent.jsx):

```
import { useState } from "react";

function Greeting({ name }) {
  return <h2>Hello, {name}!</h2>;
}

function FunctionalCounter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h3>Functional Component Counter</h3>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

export { Greeting, FunctionalCounter };
```

Code (src/ClassComponent.jsx):

```
import { Component } from "react";

class ClassCounter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  render() {
    return (
      <div>
        <h3>Class Component Counter</h3>
        <p>Count: {this.state.count}</p>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Increment
        </button>
      </div>
    );
  }
}

export default ClassCounter;
```

Code (src/App.jsx):

```
import { Greeting, FunctionalCounter } from "./FunctionalComponent";
import ClassCounter from "./ClassComponent";

function App() {
  return (
    <div style={{ textAlign: "center" }}>
      <h1>React Components Experiment</h1>
      <Greeting name="Student" />
      <FunctionalCounter />
      <hr />
      <ClassCounter />
    </div>
  );
}

export default App;
```

Output: The browser shows the heading, a greeting "Hello, Student!", a functional counter and a class counter. Each Increment button increases its own counter independently, starting from 0. After clicking each button three times, both show `Count: 3`.

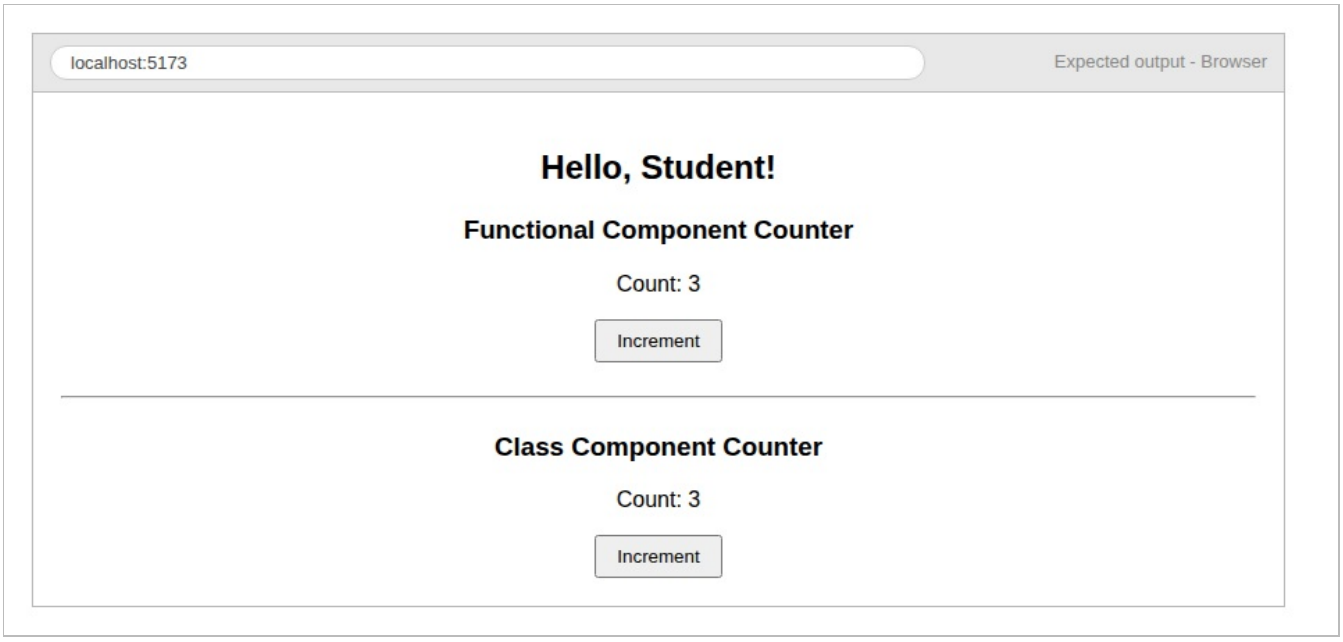


Figure: Experiment 7 - Browser output with counters

Result: Functional and class components were created in React, using props and state.

Viva questions:

1. What is the difference between props and state?
2. Why must hooks be called at the top level?
3. What is JSX?
4. Why is `setState` used instead of changing `this.state` directly?

Experiment 8: Implement HTTP Methods using JSON Server

Aim: To perform GET, POST, PUT, PATCH and DELETE operations on a REST API created with JSON Server.

Theory: JSON Server creates a full fake REST API from a JSON file with no coding. HTTP methods map to CRUD operations: GET reads data, POST creates, PUT replaces a resource, PATCH partially updates it and DELETE removes it.

Key concepts:

- **Fake REST API:** JSON Server reads `db.json` and creates routes for every top-level key. For `students` it creates `/students` (collection) and `/students/:id` (single resource).
- **HTTP methods:**

Method	Operation	Idempotent?	Typical status
GET	Read	Yes	200
POST	Create	No	201
PUT	Replace the whole resource	Yes	200
PATCH	Update only given fields	Usually yes	200
DELETE	Remove	Yes	200

- **PUT vs PATCH:** PUT sends the complete resource and replaces it. PATCH sends only the changed fields, so in the example `{"marks": 95}` leaves name and branch unchanged.
- **Idempotency:** An idempotent request gives the same result when repeated. Sending the same DELETE twice has the same final effect as sending it once.
- **Query features:** JSON Server also supports filtering and sorting, for example `/students?branch=CSE`, `/students?_sort=marks&_order=desc` and `/students?_limit=2`.
- **Content-Type header:** `Content-Type: application/json` tells the server that the body is JSON. Without it the body is not parsed.

Procedure:

1. Install JSON Server globally with `npm install -g json-server`.
2. Create `db.json` with the `students` array.
3. Start the server with `json-server --watch db.json --port 5000`.
4. Send each request in the order GET, POST, PUT, PATCH and DELETE using curl or Postman.
5. Open `db.json` after each change to confirm that the file was updated.

Setup:

```
npm install -g json-server
```

Data file (db.json):

```
{
  "students": [
    { "id": 1, "name": "Ravi", "branch": "CSE", "marks": 85 },
    { "id": 2, "name": "Priya", "branch": "ECE", "marks": 91 }
  ]
}
```

Start the server:

```
json-server --watch db.json --port 5000
```

Requests (curl; the same can be done with Postman or the browser fetch API):

```
# GET all students
curl http://localhost:5000/students

# POST a new student
curl -X POST http://localhost:5000/students -H "Content-Type: application/json" -d "
{\"name\":\"Arjun\",\"branch\":\"CSE\",\"marks\":78}"

# PUT (replace) student with id 1
curl -X PUT http://localhost:5000/students/1 -H "Content-Type: application/json" -d "{\"name\":\"Ravi
Kumar\",\"branch\":\"CSE\",\"marks\":88}"
```

```
# PATCH (partial update) student with id 2
curl -X PATCH http://localhost:5000/students/2 -H "Content-Type: application/json" -d "{\"marks\":95}"

# DELETE student with id 1
curl -X DELETE http://localhost:5000/students/1
```

Output:

- GET returns the list of students as JSON.
- POST returns the new record with an auto-generated id (3) and status 201.
- PUT returns the replaced record with id 1.
- PATCH returns student 2 with marks updated to 95 and other fields unchanged.
- DELETE returns an empty object `{}` and the record is removed from `db.json`.

Expected output - Terminal

```
$ curl http://localhost:5000/students
[{"id":1,"name":"Ravi","branch":"CSE","marks":85},
{"id":2,"name":"Priya","branch":"ECE","marks":91}]
$ curl -X POST .../students -d '{"name":"Arjun",...}'
{"name":"Arjun","branch":"CSE","marks":78,"id":3}   (201 Created)
$ curl -X PUT .../students/1 -d '{...}'
{"name":"Ravi Kumar","branch":"CSE","marks":88,"id":1}
$ curl -X PATCH .../students/2 -d '{"marks":95}'
{"id":2,"name":"Priya","branch":"ECE","marks":95}
$ curl -X DELETE .../students/1
{}
```

Figure: Experiment 8 - Terminal output of HTTP methods

Result: All five HTTP methods were implemented and verified against a JSON Server REST API.

Viva questions:

1. What is the difference between PUT and PATCH?
2. What does idempotent mean? Which methods are idempotent?
3. Which status code is returned after a successful POST, and why?
4. Why is the `Content-Type` header needed when sending JSON?

Experiment 9: Develop CRUD Operations using MongoDB

Aim: To perform Create, Read, Update and Delete operations on a MongoDB database, first from the MongoDB shell (mongosh) and then from a Node.js application using Mongoose.

Theory: MongoDB is a NoSQL document database that stores data as BSON documents inside collections. A database holds collections, and a collection holds documents. Mongoose is an Object Data Modelling (ODM) library for Node.js that defines schemas and provides an easy API for MongoDB operations. CRUD corresponds to `insertOne()`/`find()`, `updateOne()` and `deleteOne()` in the shell, and to `create()`, `find()`/`findById()`, `updateOne()`/`findByIdAndUpdate()` and `deleteOne()`/`findByIdAndDelete()` in Mongoose.

Key concepts:

- **Database hierarchy:** A MongoDB server contains databases, a database contains collections, and a collection contains documents. A document is a set of key-value pairs stored as BSON (binary JSON).
- **Schema-less design:** Documents in the same collection do not need identical fields. Mongoose adds structure through schemas.
- **_id field:** Every document gets a unique `_id` of type ObjectId when it is created. It is used to find, update and delete that document.
- **Query operators:**

Operator	Meaning	Example
`$eq`	Equal (default)	`{ branch: { $eq: "CSE" } }`
`$gt`, `$gte`	Greater than, greater than or equal	`{ marks: { $gt: 80 } }`
`$lt`, `$lte`	Less than, less than or equal	`{ marks: { $lt: 50 } }`
`$in`	Matches any value in a list	`{ branch: { $in: ["CSE", "ECE"] } }`
`$set`	Sets a field in an update	`{ $set: { marks: 90 } }`
`$inc`	Adds a number to a field	`{ $inc: { marks: 5 } }`
`$unset`	Removes a field	`{ $unset: { status: "" } }`

- **Mongoose:** A schema describes the shape and validation rules of documents (`required: true` rejects a document with no name). A model, such as `Student`, is used to run queries, and Mongoose maps it to the plural collection name `students`.
- **Connection:** `mongoose.connect()` returns a promise. The `.then` and `.catch` handlers report success or failure, and `mongoose.disconnect()` closes the connection when the script ends.
- **Shell vs code:** The mongosh commands and the Mongoose methods perform the same operations. The shell is useful for checking data, and Mongoose is used inside applications.

Procedure:

1. Start the MongoDB service, then open a terminal and run `mongosh`.
2. Create `collegeDB` with `use collegeDB` and insert three students with `insertOne` and `insertMany`.
3. Run the READ queries with filters and operators, then update one document and delete one.
4. Create the Node.js project, install Mongoose and write `crud.js`.
5. Run `node crud.js` and compare the output with the expected output shown above.

Part A: Connecting to MongoDB and CRUD using the Shell (mongosh)

Step 1: Start the MongoDB server. (Installed as a Windows service, MongoDB normally starts automatically. To start it manually from the install folder:)

```
mongod --dbpath "C:\data\db"
```

Step 2: Connect using the shell. Open a new terminal and run:

```
mongosh
```

To connect to a specific database directly, or to a remote server or Atlas cluster:

```
mongosh "mongodb://127.0.0.1:27017/collegeDB"
mongosh "mongodb+srv://<username>:<password>@<cluster-url>/collegeDB"
```

Step 3: Basic database commands.

```
show dbs                  // list all databases
use collegeDB             // switch to (or create) the collegeDB database
show collections          // list collections in the current database
db                        // print the current database name
```

Step 4: CRUD commands.

```
// CREATE
db.students.insertOne({ name: "Ravi", branch: "CSE", marks: 85 })
db.students.insertMany([
  { name: "Priya", branch: "ECE", marks: 91 },
  { name: "Arjun", branch: "CSE", marks: 78 }
])

// READ
db.students.find()                          // all documents
db.students.find({ branch: "CSE" })         // filter by field
db.students.find({ marks: { $gt: 80 } })    // marks greater than 80
db.students.findOne({ name: "Ravi" })       // first matching document

// UPDATE
db.students.updateOne({ name: "Ravi" }, { $set: { marks: 90 } })
db.students.updateMany({ branch: "CSE" }, { $set: { status: "active" } })

// DELETE
db.students.deleteOne({ name: "Ravi" })
db.students.deleteMany({ marks: { $lt: 50 } })
```

Step 5: Clean up and exit.

```
db.students.drop()        // delete the whole collection (optional)
db.dropDatabase()         // delete the current database (optional)
exit                      // leave mongosh
```

Shell output (sample):

```
test> use collegeDB
switched to db collegeDB
collegeDB> db.students.insertOne({ name: "Ravi", branch: "CSE", marks: 85 })
{ acknowledged: true, insertedId: ObjectId('...') }
collegeDB> db.students.find({ branch: "CSE" })
[ { _id: ObjectId('...'), name: 'Ravi', branch: 'CSE', marks: 85 } ]
collegeDB> db.students.updateOne({ name: "Ravi" }, { $set: { marks: 90 } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
collegeDB> db.students.deleteOne({ name: "Ravi" })
{ acknowledged: true, deletedCount: 1 }
```

Part B: CRUD using Node.js and Mongoose

Setup:

```
mkdir mongo-crud
cd mongo-crud
npm init -y
npm install mongoose
```

Make sure MongoDB is running locally on `mongodb://127.0.0.1:27017`, or use MongoDB Atlas and replace the connection string.

Code (crud.js):

```
const mongoose = require("mongoose");

mongoose.connect("mongodb://127.0.0.1:27017/collegeDB")
  .then(() => console.log("Connected to MongoDB"))
  .catch(err => console.error("Connection error:", err));

// Define schema and model
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  branch: String,
  marks: Number
```

```
});
const Student = mongoose.model("Student", studentSchema);

async function runCrud() {
  // CREATE
  const s1 = await Student.create({ name: "Ravi", branch: "CSE", marks: 85 });
  console.log("Created:", s1);

  // READ
  const all = await Student.find();
  console.log("All students:", all);

  // UPDATE
  await Student.updateOne({ name: "Ravi" }, { marks: 90 });
  console.log("Updated marks for Ravi");

  // DELETE
  await Student.deleteOne({ name: "Ravi" });
  console.log("Deleted Ravi");

  await mongoose.disconnect();
}

runCrud();
```

Output:

```
Connected to MongoDB
Created: { name: 'Ravi', branch: 'CSE', marks: 85, _id: new ObjectId('...'), __v: 0 }
All students: [ { _id: ..., name: 'Ravi', branch: 'CSE', marks: 85, __v: 0 } ]
Updated marks for Ravi
Deleted Ravi
```

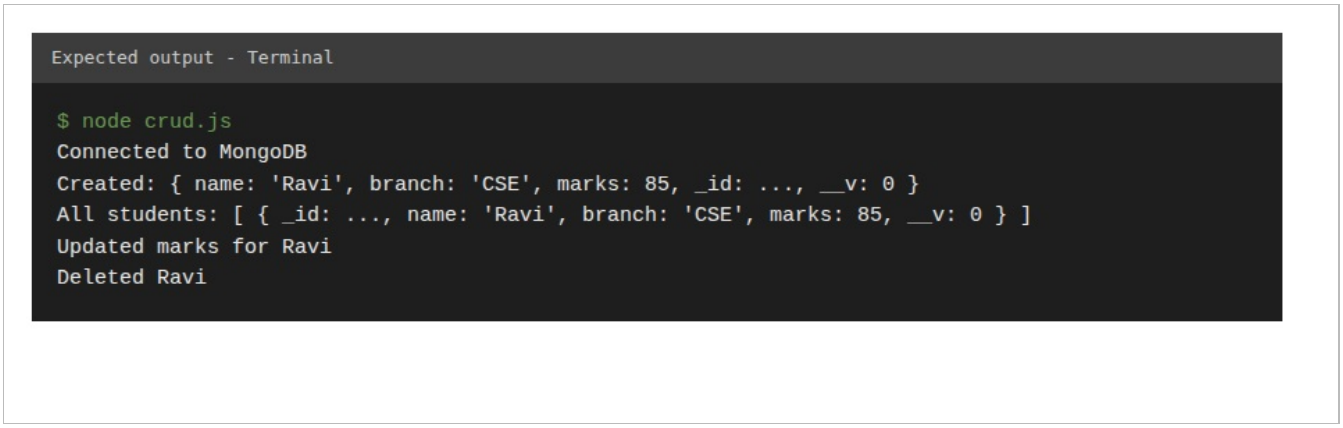


Figure: Experiment 9 - Terminal output of crud.js

Result: Create, Read, Update and Delete operations were performed on a MongoDB collection using the mongosh shell and using Mongoose.

Viva questions:

1. What is the difference between a collection and a document?
2. What is the purpose of `_id`?
3. What is the difference between `updateOne` and `updateMany`?
4. What is a Mongoose schema and why is it useful?

Experiment 10: Develop Main Project using Express, React and MongoDB

Aim: To develop a full-stack application (a Student Management System) using Express for the backend, React for the frontend and MongoDB for the database.

Theory: A full-stack MERN-style application separates concerns into three layers. MongoDB stores data, Express with Node.js exposes REST APIs, and React consumes those APIs to render and manage the user interface. The frontend communicates with the backend over HTTP using `fetch` or Axios, and CORS must be enabled so that the React app on a different port can call the server.

Key concepts:

- **MERN stack:** MongoDB (database), Express (server), React (user interface) and Node.js (runtime). Each layer communicates only through well-defined interfaces, which makes each part easy to change.
- **Request flow:**

```
React (localhost:5173)  --HTTP JSON-->  Express API (localhost:5000)  --Mongoose-->  MongoDB (27017)
       ^                                          |
       +---------------- JSON response ---------+
```

- **CORS:** Browsers block a page from calling a server on a different origin (different port counts as different). `cors()` adds headers that allow the React app to make these requests.
- **Asynchronous code:** Database calls take time, so they are written with `async` and `await`. The route waits for the database result before sending the response.
- **Frontend state:** `useState` stores the list of students, the form values and the current input. `useEffect(() => ..., [])` runs `loadStudents` once when the page first loads.
- **Controlled inputs:** Each form field's `value` comes from state and `onChange` updates the state. This keeps the screen and the data in step.
- **Keys in lists:** `key={s._id}` lets React track each row, so it updates the correct row when the list changes.
- **Ports:** The API runs on port 5000 and the Vite app on 5173, so the URL in `App.jsx` must point to the API address.

Procedure:

1. Start MongoDB, then run `node server.js` in the `server` folder and confirm `MongoDB connected`.
2. Create the Vite React project in `client`, replace `App.jsx` with the code above, and run `npm run dev`.
3. Open `http://localhost:5173` and add a student with a name, branch and marks.
4. Confirm that the new row appears in the table and in MongoDB (check with `db.students.find()` in mongosh).
5. Click Delete on a row and confirm that it disappears from the table and the database.

Project structure:

```
student-management/
├── server/
│   ├── server.js
│   └── package.json
└── client/            (created with Vite + React)
    └── src/App.jsx
```

Backend setup:

```
mkdir student-management && cd student-management
mkdir server && cd server
npm init -y
npm install express mongoose cors
```

Code (server/server.js):

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.error(err));
```

```
const Student = mongoose.model("Student", new mongoose.Schema({
  name: { type: String, required: true },
  branch: String,
  marks: Number
}));

// Read all students
app.get("/api/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// Create a student
app.post("/api/students", async (req, res) => {
  const student = await Student.create(req.body);
  res.status(201).json(student);
});

// Update a student
app.put("/api/students/:id", async (req, res) => {
  const student = await Student.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(student);
});

// Delete a student
app.delete("/api/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({ message: "Deleted" });
});

app.listen(5000, () => console.log("Server running on http://localhost:5000"));
```

Frontend setup:

```
cd ..
npm create vite@latest client -- --template react
cd client
npm install
npm run dev
```

Code (client/src/App.jsx):

```
import { useState, useEffect } from "react";

const API = "http://localhost:5000/api/students";

function App() {
  const [students, setStudents] = useState([]);
  const [form, setForm] = useState({ name: "", branch: "", marks: "" });

  const loadStudents = async () => {
    const res = await fetch(API);
    setStudents(await res.json());
  };

  useEffect(() => { loadStudents(); }, []);

  const addStudent = async (e) => {
    e.preventDefault();
    await fetch(API, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ ...form, marks: Number(form.marks) })
    });
    setForm({ name: "", branch: "", marks: "" });
    loadStudents();
  };

  const deleteStudent = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    loadStudents();
  };

  return (
```

```
    <div style={{ fontFamily: "Arial", padding: 20 }}>
      <h1>Student Management System</h1>

      <form onSubmit={addStudent}>
        <input placeholder="Name" value={form.name}
          onChange={e => setForm({ ...form, name: e.target.value })} />
        <input placeholder="Branch" value={form.branch}
          onChange={e => setForm({ ...form, branch: e.target.value })} />
        <input placeholder="Marks" type="number" value={form.marks}
          onChange={e => setForm({ ...form, marks: e.target.value })} />
        <button type="submit">Add Student</button>
      </form>

      <table border="1" cellPadding="8" style={{ marginTop: 20 }}>
        <thead>
          <tr><th>Name</th><th>Branch</th><th>Marks</th><th>Action</th></tr>
        </thead>
        <tbody>
          {students.map(s => (
            <tr key={s._id}>
              <td>{s.name}</td>
              <td>{s.branch}</td>
              <td>{s.marks}</td>
              <td><button onClick={() => deleteStudent(s._id)}>Delete</button></td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Output: The backend prints `MongoDB connected` and `Server running on http://localhost:5000`. The React page (http://localhost:5173) shows a form for adding students and a table listing all students stored in MongoDB. Adding a student inserts it into the table immediately, and clicking Delete removes that row from both the table and the database.

Expected output - Terminal

```
$ node server.js
MongoDB connected
Server running on http://localhost:5000
```

Figure: Experiment 10 - Terminal output

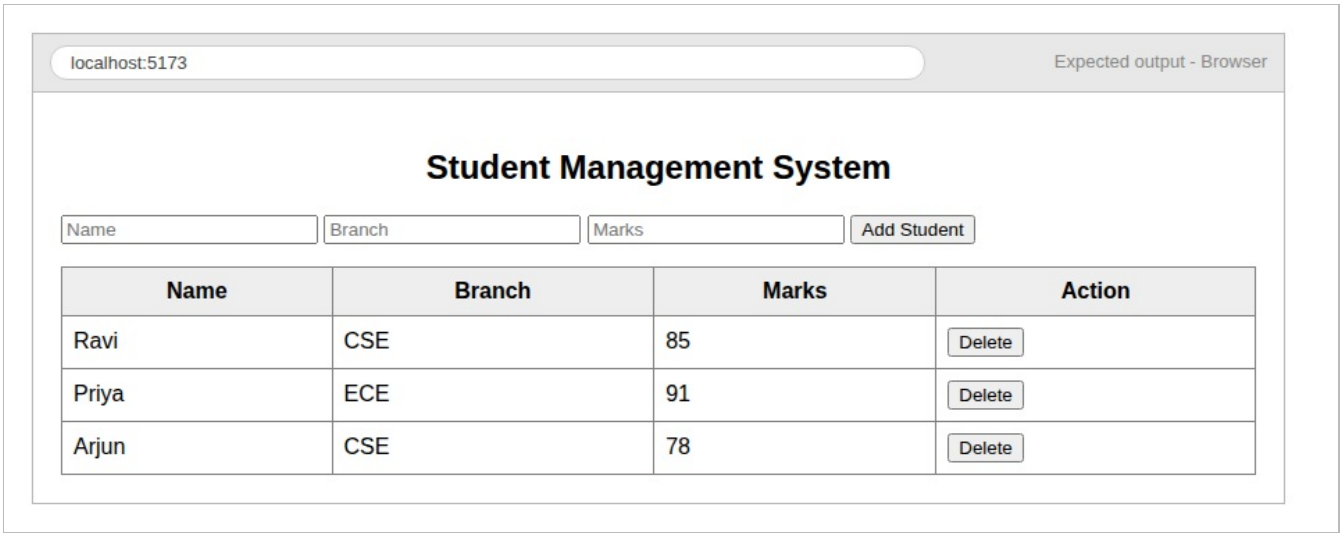


Figure: Experiment 10 - Browser output of Student Management System

Result: A full-stack Student Management System was developed using Express, React and MongoDB, with working create, read, update and delete functionality.

Viva questions:

1. What do the letters M, E, R and N stand for in MERN?
2. Why is CORS needed when React and Express run on different ports?
3. Why is `key` required when rendering a list in React?
4. Why does the `useEffect` in this application have an empty dependency array?

Conclusion

All ten experiments were completed, covering static and dynamic front-end design with HTML, CSS and JavaScript (CO1); server-side development with Node.js and Express (CO2); React components and REST API testing with JSON Server (CO3); and MongoDB CRUD operations through both the mongosh shell and Mongoose, culminating in a full-stack main project (CO4).